

# BASES DE DONNÉES

---

CM :

*Dénormalisation : tableaux, types UDT, héritage  
de tables, types JSON/JSONB*

Vincent COUTURIER

Maitre de conférences – Université Savoie Mont-Blanc

[vincent.couturier@univ-smb.fr](mailto:vincent.couturier@univ-smb.fr)

# NORME SQL:1999

---

# SQL:1999

- Types distincts
- Types utilisateurs (UDT)
- Tables typées
- Héritage :
  - Hiérarchie de types
  - Hiérarchie de tables
- Méthodes associées aux UDT

Norme respectée dans Informix.

Dans Oracle ou PostgreSQL : syntaxe parfois (souvent !) différente.

# POSTGRESQL

---

# SGBD Objet Relationnel

- Tableaux
- Types définis par l'utilisateur (UDT)
- Héritage de tables
- Types géométriques (non abordés ici !)

# Tableaux

- Possibilité de définir des tableaux à plusieurs dimensions
- On peut fixer ou pas la taille des tableaux
- **Utilisables sur des FK mais cela complexifie les jointures (donc à éviter !)**
- Exemple :

```
create table groupe (
  numg integer primary key,
  nom varchar(20) not null,
  membres integer[]);
```

```
insert into groupe values (1, 'clients', '{2, 4, 6}');
```

OU

```
insert into groupe values (1, 'clients', ARRAY[2, 4, 6]);
```

```
select membres[1] as premier_membre from groupe;
```

```
select premier_membre as deux_dern. deux_derniers_membres from groupe;
```

|   | premier_membre<br>integer |
|---|---------------------------|
| 1 | 2                         |

|   | deux_derniers_membres<br>integer[] |
|---|------------------------------------|
| 1 | {4,6}                              |

# Types composites : LDD

- Type composite (Row types) = Type défini par l'utilisateur (User defined Data Type - UDT)
- LDD :

```
DROP TYPE IF EXISTS adresse_t CASCADE;
CREATE TYPE adresse_t AS (
    rue varchar(200),
    ville varchar(80),
    cp char(5));
```

```
DROP TABLE IF EXISTS personne CASCADE;
CREATE TABLE personne (
    nom varchar(100) PRIMARY KEY,
    prenoms varchar(20)[],
    adr_perso adresse_t,
    adr_pro adresse_t);
```

*Remarque : les tableaux de type composite sont supportés depuis la version 12 (ex. `adresse_t []`). Il est préférable ici d'utiliser 2 variables différentes afin de ne pas confondre les 2 types d'adresse.*

# Types composites : LMD

```
INSERT INTO personne VALUES ('Dupont',  
    ARRAY ['Jean', 'Robert'],  
    ROW ('rue des Amandiers', 'Toulon', '83100'),  
    ROW ('rue des Mimosas', 'Marseille', '13009'));
```

```
UPDATE personne  
SET adr_perso =  
    ROW ('rue des Mimosas', 'Marseille', '13009')  
WHERE nom = 'Dupont';
```

```
UPDATE personne  
SET adr_pro.ville = 'Toulon',  
    adr_pro.cp = '83100'  
WHERE nom = 'Dupont';
```



# Types composites : LID

```
SELECT (adr_pro).ville -- Ne pas oublier les ( )  
FROM personne  
WHERE nom = 'Dupont';
```

```
SELECT (p.adr_pro).ville  
FROM personne p  
WHERE p.nom='Dupont';
```

```
SELECT nom, prenom[1]  
FROM personne p  
WHERE (p.adr_perso).ville = 'Toulon';
```

# Héritage

- Héritage simple de table
- Héritage multiple (rare !)

# Héritage

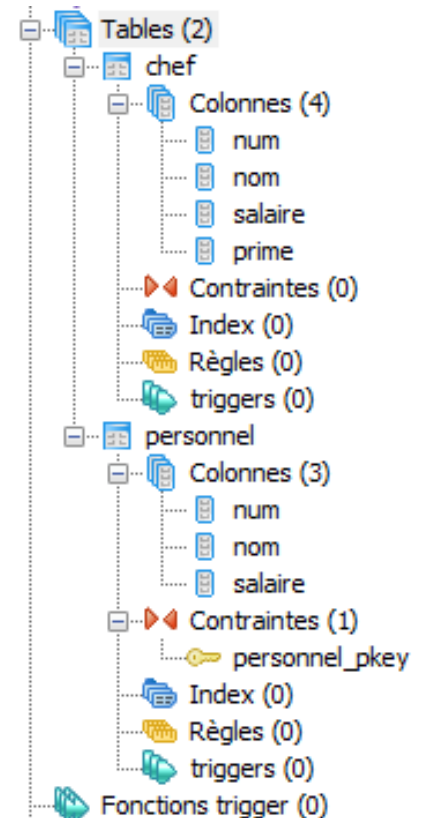
```
create table personnel (
    num integer primary key,
    nom varchar(50) not null,
    salaire float);
```

```
create table chef (prime float)
    inherits (personnel) ;
```

```
insert into personnel values (1, 'dupont',
7000.0);
```

```
insert into chef values (2, 'durant',
10000.0, 2000.0);
```

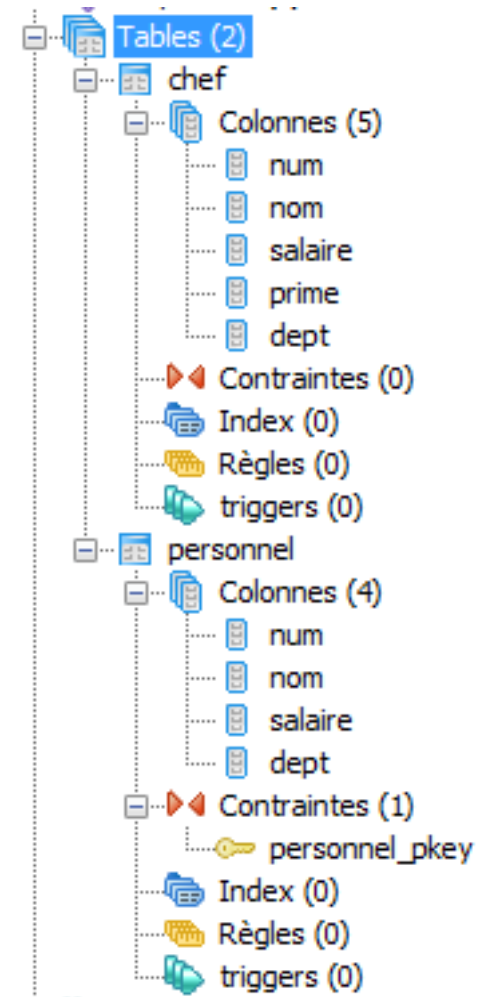
```
select * from personnel;
```



|   | num<br>integer | nom<br>character varying(50) | salaire<br>double precision |
|---|----------------|------------------------------|-----------------------------|
| 1 | 1              | dupont                       | 7000                        |
| 2 | 2              | durant                       | 10000                       |

# Héritage

```
Alter table personnel  
add column dept integer;
```



# Héritage

```
SELECT p.relname, pe.nom  
FROM personnel pe  
JOIN pg_class p ON pe.tableoid = p.oid;
```

|   | relname<br>name | nom<br>character varying(50) |
|---|-----------------|------------------------------|
| 1 | personn         | dupont                       |
| 2 | chef            | durant                       |

# Exercices

1. Créer un type `zonegeo_t` (`CP`, `ville`)
2. Créer une table `Personne` (`nom`, `prenom`, `rue`, `zonegeo` de type `zonegeo_t`)
3. Montrer l'insertion de données dans la table `Personne`
4. Ecrire une requête SQL affichant les personnes (`nom`, `prenom`, `ville`) habitant dans le Rhône.
5. Créer une table `Etudiant` (`num_etudiant`) héritant de `personne`
6. Montrer l'insertion de données dans la table `Etudiant`

# JSON

---

# Présentation

- PostgreSQL intègre des fonctionnalités NoSQL (*Not Only SQL*) depuis la version 9.3 :
  - Version 9.3 : Type de données JSON ⇔ Clé/valeur
    - 1 clé -> 1 document JSON
    - Le document JSON peut-être affiché mais son contenu n'est pas adressable (les champs ne sont pas requêtables).
  - Version 9.4 : Type de données JSONB ⇔ orienté-documents
    - 1 clé -> 1 Document JSON (stocké au format binaire)
    - Chaque champ du JSON est adressable (« requêtable ») par une clé.
- Il est ainsi possible :
  - D'ajouter un (ou plusieurs) champ supplémentaire dans une table qui contiendra un document JSON.
  - Permet de mixer relationnel et NoSQL (JSON)



# Différences JSON/JSONB

- La différence majeure réside dans **l'efficacité** :
  - Le type de données JSON stocke une copie exacte du texte en entrée, que chaque fonction doit analyser à chaque exécution.
  - Alors que le type de données JSONB est stocké dans un format binaire décomposé (c'est-à-dire non pas comme une chaîne ASCII/UTF-8, mais comme un code binaire)
    - => Cela rend l'insertion légèrement plus lente du fait du surcoût de la conversion, mais est significativement plus rapide pour traiter les données, puisqu'aucune analyse n'est nécessaire.
- Le type JSONB gère **l'indexation** des données des champs du fichier JSON (le contenu du JSON est donc requêtable grâce aux champs).
- Conservation des espaces et duplication des clés :
  - Comme le type JSON stocke une copie exacte du texte en entrée, il conservera les espaces sémantiquement non significatifs entre les jetons, ainsi que l'ordre des clés au sein de l'objet JSON.
  - Si un objet JSON contient dans sa valeur la même clé plus d'une fois, toutes les paires clé/valeur sont conservées (les fonctions de traitement considèrent la dernière clé comme celle significative).
  - À l'inverse, JSONB ne conserve ni les espaces non significatifs, ni l'ordre des clés d'objet (les clés sont triées), ni ne conserve les clés d'objet dupliquées. Si des clés dupliquées sont présentées en entrée, seule la dernière clé est conservée.

# Exemple : type de données JSON (clé/valeur)

```
CREATE TABLE ClientJSON (  
    id integer,  
    client json  
);  
insert into ClientJSON values (1,  
    '{  
        "nom" : "DURAND",  
        "prenom" : "Jean",  
        "societe" : false,  
        "adresse" : "12 rue de la Gare, 74000, Annecy",  
        "categories" : ["EURL", "Informatique"]  
    }');  
insert into ClientJSON values (2,  
    '{  
        "raison sociale" : "INFORMATIQUE SA",  
        "societe" : true,  
        "adresse" : "9 rue de la Gare, 74000, Annecy",  
        "categories" : ["Grande entreprise", "Informatique"]  
    }');
```

Fonctionne, mais contre-performant : on ne stocke dans un type JSON que des données qui seront peu souvent requêtées car moins rapide d'accès que les types scalaires (CHAR, VARCHAR, etc.)

## Exemple : type de données JSONB (orienté document)

```
CREATE TABLE ClientJSONB (  
    id integer,  
    client jsonb  
);  
insert into ClientJSONB values (1,  
    '{  
        "nom" : "DURAND",  
        "prenom" : "Jean",  
        "societe" : false,  
        "adresse" : "12 rue de la Gare, 74000, Annecy",  
        "categories" : ["EURL", "Informatique"]  
    }');  
insert into ClientJSONB values (2,  
    '{  
        "raison sociale" : "INFORMATIQUE SA",  
        "societe" : true,  
        "adresse" : "9 rue de la Gare, 74000, Annecy",  
        "categories" : ["Grande entreprise", "Informatique"]  
    }');
```

IDEM. Fonctionne, mais contre-performant.

# Exemple : type de données JSONB (orienté document)

```
select * from ClientJSONB;
```

| id<br>integer | client<br>json                                                                                                                                    |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 1             | {"nom":"DURAND","adresse":"12 rue de la Gare, 74000, Annecy","prenom":"Jean","categories":["EURL","Informatique"],"societe":false}                |
| 2             | {"raison sociale":"INFORMATIQUE SA","adresse":"9 rue de la Gare, 74000, Annecy","societe":true,"categories":["Grande entreprise","Informatique"]} |

```
SELECT client->'nom', client->'prenom' FROM clientJSONB WHERE  
client @> '{"nom": "DURAND"}';
```

|   | ?column?<br>jsonb | ?column?<br>jsonb |
|---|-------------------|-------------------|
| 1 | "DURAND"          | "Jean"            |

⇒ Possible de rechercher dans le contenu d'un type JSONB. Syntaxe à utiliser pour requêter un JSONB : <https://docs.postgresql.fr/18/functions-json.html>

Il est nécessaire de renommer les colonnes :

```
SELECT client->'nom' as nom, client->'prenom' as prenom FROM  
clientJSONB WHERE client @> '{"nom": "DURAND"}';
```

|   | nom<br>jsonb | prenom<br>jsonb |
|---|--------------|-----------------|
| 1 | "DURAND"     | "Jean"          |

# JSONB : avantages et inconvénients

- + :
  - Efficace
  - Traitement beaucoup plus rapide que JSON
  - Supporte l'indexation
  - Des schémas plus simples (remplacement des tables entités-attributs-valeurs (EAV)) par des colonnes JSONB, qui peuvent être interrogées, indexées et jointes, ce qui permet d'améliorer les performances jusqu'à 1000 fois !
- - :
  - Un coût plus important en termes de stockage que des champs scalaires (la taille disque peut être doublée).
    - JSONB peut également prendre plus d'espace disque que le JSON simple en raison d'une empreinte de table plus grande, mais pas toujours,
  - Une insertion légèrement plus lente que JSON (en raison de la surcharge de conversion ajoutée),
  - Certaines requêtes (en particulier les requêtes agrégées) peuvent être plus lentes en raison de l'absence de statistiques sur les tables :
    - Pour toute colonne scalaire, PostgreSQL enregistre des statistiques descriptives telles que le nombre de valeurs distinctes et les plus communes, la fraction de valeurs NULL, et, pour les types ordonnés, un histogramme de la distribution des données.
    - Ces informations ne sont pas disponibles pour les champs JSON/JSONB. Cela engendrera une pénalité importante de performance (même si chaque nouvelle version de PostgreSQL a apporté une amélioration des performances), en particulier lors de l'agrégation de données (COUNT, AVG, SUM, etc.) sur des champs JSON.
  - Pour plus de détails pour savoir quand ne pas utiliser un type JSONB :  
<https://heap.io/blog/when-to-avoid-jsonb-in-a-postgresql-schema>

# JSON/JSONB : conseils

- A utiliser pour ajouter des attributs arbitraires et variables (facultatifs) aux tables, liés à des exigences souples.
- Adapté aux colonnes qui ne sont pas consultées fréquemment.
- MAIS même dans les applications où on désire le maximum de flexibilité, il est toujours recommandé que les documents JSON aient une structure quelque peu fixée.
- Si vous travaillez uniquement avec la représentation JSON dans une application, PostgreSQL n'est utilisé que pour stocker et récupérer cette représentation => utiliser le type JSON.
- OU Si vous faites beaucoup d'opérations sur le document JSON dans PostgreSQL ou si vous utilisez l'indexation sur un champ JSON => utiliser le type JSONB.

# Expressions de chemin SQL/JSON

- Les opérateurs @>, -> ou ->> ne sont pas des opérateurs standards, mais propres à PostgreSQL.
- Les expressions SQL/JSON (ou *SQL/JSON path expressions*) et les opérateurs associés ont été ajoutés dans la norme SQL:2016 (dernière version) et permettent de requêter les données JSON (opérateurs standards).
- Spécifient les éléments à extraire des données JSON.
- Supportés depuis PostgreSQL 12
- <https://docs.postgresql.fr/18/functions-json.html#FUNCTIONS-SQLJSON-PATH>

## SQL/JSON standard conformance

| SQL/JSON feature                | PostgreSQL 12 | Oracle 18c | MySQL 8.0.4 | MS SQL Server 2017 |
|---------------------------------|---------------|------------|-------------|--------------------|
| JSON_OBJECT: 7                  | 6             | 5          | 1           | 0                  |
| JSON_ARRAY: 4                   | 4             | 4          | 1           | 0                  |
| JSON_OBJECTAGG<br>JSON_ARRAYAGG | 2             | 2          | 2           | 0                  |
| RETURNING: 3                    | 3             | 2          | 2           | 0                  |
| FORMAT JSON: 1                  | 1             | 1          | 0           | 0                  |
| IS JSON: 2                      | 2             | 1          | 0           | 0                  |
| JSON_EXISTS: 5                  | 5             | 5          | 0           | 1                  |
| JSON_QUERY: 3                   | 3             | 3          | 0           | 1                  |

## SQL/JSON standard conformance

| SQL/JSON feature | PostgreSQL 12 | Oracle 18c   | MySQL 8.0.4  | MS SQL Server 2017 |
|------------------|---------------|--------------|--------------|--------------------|
| JSON PATH: 15    | 15            | 11           | 5            | 2                  |
| TOTAL: 42        | <b>41/42</b>  | <b>34/42</b> | <b>11/42</b> | <b>4/42</b>        |

PostgreSQL 12 could have **the best implementation** of SQL/JSON standard !

SQL/JSON Standard Conformance in PostgreSQL, Oracle, MySQL, SQL Server  
<http://obartunov.livejournal.com/200076.html>

<https://obartunov.livejournal.com/207356.html>

# Expressions de chemin SQL/JSON : exemple

- Soit le JSON suivant de suivi GPS :

```
{
  "track": {
    "segments": [
      {
        "location": [ 47.763, 13.4034 ],
        "start time": "2018-10-14 10:05:14",
        "HR": 73
      },
      {
        "location": [ 47.706, 13.2635 ],
        "start time": "2018-10-14 10:39:21",
        "HR": 135
      }
    ]
  }
}
```

Pour récupérer les segments de track disponibles, vous utiliserez l'opérateur accesseur ainsi :  
'\$.track.segments'

Pour récupérer un élément qui est un tableau : '\$.track.segments[\*].location'

Et pour récupérer une valeur du dit tableau : '\$.track.segments[0].location'

etc.

Dans PostgreSQL, il faut utiliser la fonction `jsonb_path_query(json, jsonpath)` pour requêter les documents JSONB avec des expressions json path SQL (expressions SQL/JSON).